Indoor Mobile Robot Navigation:

Cave Robot

Artem Pugachuk

Arbi Kullakshi

European University of Tirana

Course Project Research Paper

June 7, 2026

Abstract

This paper presents Cave Robot, an autonomous indoor navigation system designed to move through procedurally generated cave-like environments, build an occupancy map from LiDAR observations, and return to its starting point using the map produced during the mission. The project addresses three linked problems in mobile robotics: path planning in constrained spaces, localization under motion uncertainty, and mapping from range measurements. The implementation combines a cellular-automata cave generator, A* search on a clearance-weighted three-dimensional grid, an Extended Kalman Filter for pose estimation, and occupancy-grid SLAM based on LiDAR ray updates. The navigation core is implemented in Rust and connected to ROS 2 and Gazebo through a JSON pipe bridge. A fixed-seed offline simulation with a 64 x 32 x 4 voxel cave completed a full start-to-goal-to-start mission in 294 steps, using a 22-cell forward path and an 18-cell return path. The results show that classical robotics methods can be integrated into a coherent simulation pipeline for indoor autonomous navigation, while also revealing limitations related to map prior assumptions, simulated sensing, and localization realism.

Keywords: autonomous navigation, A search, Extended Kalman Filter, SLAM, LiDAR, Gazebo, ROS 2*

Table of Contents

Introduction 4

Literature Review and Theoretical Background 5

Methodology 7

System Implementation 8

Results 10

Discussion 11

Conclusion 12

References 13

Appendix A: Reproducibility Notes 14

Introduction

Indoor mobile robot navigation is a central problem in autonomous systems because it requires a robot to make useful movement decisions without relying on the open space, stable satellite localization, or simple geometry that may exist in outdoor environments. Caves, tunnels, basements, industrial corridors, and damaged structures create especially difficult conditions: passages can be narrow, visibility is limited, obstacles are frequent, and a reliable map may not be available before the mission begins. A robot operating in this type of space must therefore combine planning, localization, and mapping rather than treating them as isolated modules.

The Cave Robot project studies this problem through a simulated autonomous drone placed inside a procedurally generated cave environment. The mission is defined as a round trip. First, the robot travels from a start cell to a goal cell using a complete graph generated from the known cave. During this forward trip, the robot observes the environment with LiDAR and updates an occupancy map. After reaching the goal, it replans from the goal back to the start using the SLAM-derived map rather than the original ground-truth grid. This return leg is important because it tests whether the robot can act on the map it has built from perception.

The objective of this research paper is to describe the problem, theoretical background, methodology, implementation, simulation results, and limitations of the Cave Robot system. The work is framed as a course project in autonomous systems modeling and simulation. It emphasizes clear integration of classical robotics methods: graph search for path planning, an Extended Kalman Filter for localization, and occupancy-grid mapping for perception-based navigation.

Literature Review and Theoretical Background

The planning component of Cave Robot is based on A* search, a heuristic graph search algorithm introduced by Hart, Nilsson, and Raphael (1968). A* evaluates each candidate node with $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost accumulated so far and $h(n)$ is an estimate of the remaining cost to the goal. When the heuristic is admissible, A* can produce an optimal path while searching fewer nodes than uniform-cost search. In the Cave Robot implementation, the graph is a three-dimensional cardinal-direction grid, and the cost of each cell is adjusted by clearance from walls.

Localization is treated with an Extended Kalman Filter (EKF), a common estimator for systems whose motion or measurement models are nonlinear. The Kalman filter family estimates a hidden state by alternating between prediction and correction. The EKF extends this idea by linearizing nonlinear functions around the current estimate (Welch & Bishop, 2006). Cave Robot tracks x, y, z, and yaw. Motor commands provide the prediction step, while Gazebo pose measurements can be used as a correction source when available.

Mapping follows the occupancy-grid tradition introduced by Elfes (1989), where the environment is divided into cells and each cell stores a belief about whether it is occupied. In Cave Robot, LiDAR rays are traced through the grid. Endpoints increase occupied evidence, while intermediate cells receive free-space evidence. This creates a map that can be converted into a passable grid for return planning.

SLAM, or simultaneous localization and mapping, is the broader problem of estimating both the robot trajectory and the map from sensor data. Probabilistic Robotics by Thrun, Burgard, and Fox (2005) formalizes many of the particle-filter and Bayesian estimation ideas behind this class of systems. The current Cave Robot implementation includes a particle-filter

SLAM structure, but the map update is anchored to the EKF/Gazebo pose estimate for stable map construction. This is a practical design choice for simulation, but it also limits the realism of the SLAM claim.

Incremental replanning is represented by D* Lite, which is implemented in the pathfinding crate but is not used in the primary mission. D* Lite is useful when edge costs change as a robot discovers the world (Koenig & Likhachev, 2002). Since the forward path in this project is planned on a known generated cave, A* is sufficient for the current mission. D* Lite remains a future extension for fully online exploration.

Table 1

Core Algorithms Used in the Cave Robot System

Component	**Method**	**Purpose**	**Current Role**
Planning	A*	Find shortest feasible path	Used for forward and return paths
Localization	Extended Kalman Filter	Estimate x, y, z, yaw	Runs throughout navigation
Mapping	Occupancy grid	Represent free and occupied cells	Built from LiDAR rays
SLAM	Particle-filter structure	Estimate map and trajectory	Map update used; particle localization limited
Replanning	D* Lite	Repair paths when map changes	Implemented, reserved for future online use

Methodology

The project methodology is organized around a complete simulated mission. The system first generates a cave, converts it into a simulated world, places the robot at a valid start location, plans a route to the goal, updates a map during motion, and then returns to the start using the map built from observation.

The cave generator creates a 64 x 32 x 4 voxel environment by applying a cellular automaton independently to each horizontal level. Random noise is smoothed by repeated neighborhood rules, disconnected floor regions are connected by flood fill and wall breaching, and vertical traversal is added through ramps and shafts. The result is serialized as cave.json, which stores the grid, dimensions, start cell, and end cell.

For forward planning, each passable cell becomes a node in a three-dimensional grid graph. The planner excludes diagonal movement to prevent corner clipping. A clearance pass computes the Manhattan distance from each passable cell to the nearest wall. Cells adjacent to walls receive a higher traversal cost, cells one step away receive a moderate cost, and open cells receive the base cost. This weighting encourages paths through corridor centers instead of along wall boundaries.

The robot motion model is kinematic. For each command interval, the pose is advanced by x' = x + v cos(theta) dt, y' = y + v sin(theta) dt, z' = z + vz dt, and theta' = theta + omega dt. The EKF uses this model for prediction and applies measurement correction when an external pose observation is available. The covariance grows with movement, representing increasing uncertainty from dead reckoning.

Mapping is performed by ray tracing LiDAR measurements into a log-odds occupancy grid. A ray endpoint increases evidence that a cell is occupied, while cells crossed before the

endpoint receive free-space evidence. After the forward journey, the occupancy grid is thresholded into a passable graph. A* then replans from the goal back to the start on this observed map.

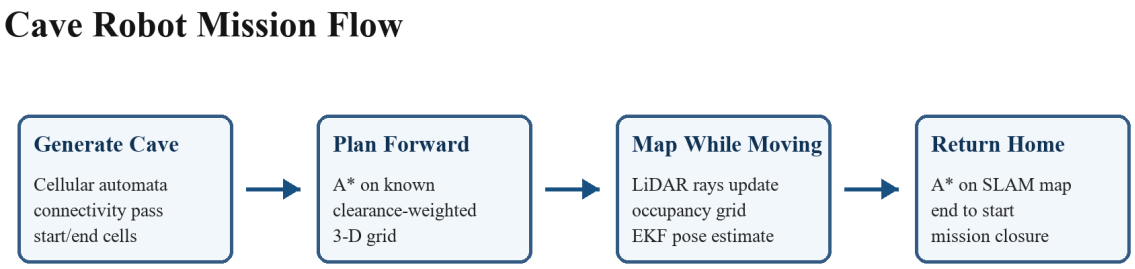

Figure 1

Mission flow from procedural generation to return navigation.

System Implementation

The implementation is organized as a Rust workspace with separate crates and applications for cave generation, shared data types, pathfinding, SLAM, Kalman filtering, and the robot executable. ROS 2 and Gazebo integration are handled through a bridge node that connects simulator topics to the robot process. The robot process can also run offline without Gazebo by using internally simulated LiDAR scans.

The generator application writes cave.json and can convert the cave grid into a Gazebo SDF world. The generated world includes merged wall geometry, floor and ceiling slabs, start and end markers, lights, and a drone model with LiDAR. The ROS 2 bridge receives LiDAR data and publishes velocity commands through Gazebo topics. The navigation core communicates through line-delimited JSON, which keeps the robotics logic independent from simulator-specific code.

The main robot loop processes each scan, predicts and corrects pose, updates the occupancy grid, checks whether the current target has been reached, and emits a velocity command. Recovery logic handles spinning and lack of progress by backing up, rotating, skipping waypoints, or replanning from the current cell. This is important in caves because tight turns and vertical transitions can make a nominal path difficult to follow exactly.

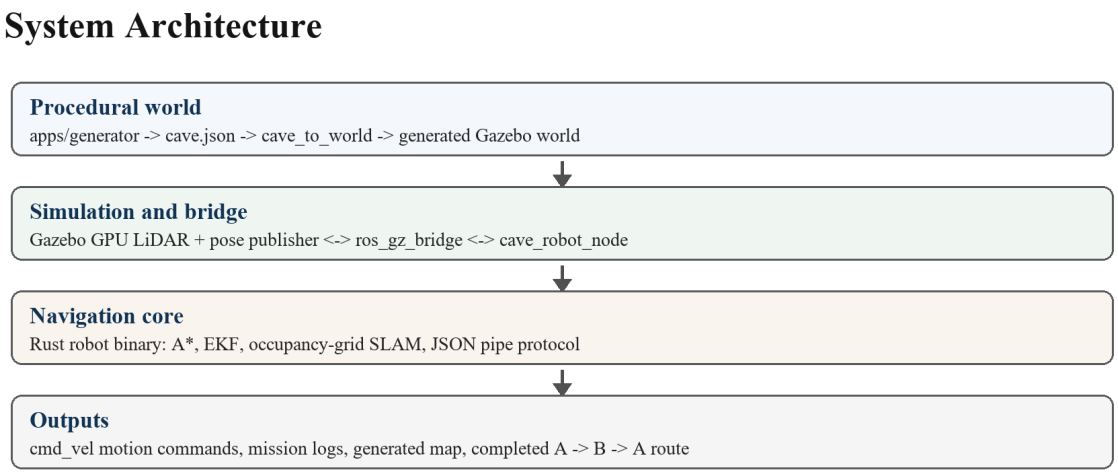

Figure 2

High-level architecture of the Cave Robot simulation stack.

Table 2

Simulation and Implementation Parameters

Parameter	Value	Purpose
Default cave size	64 x 32 x 4 cells	Three-dimensional navigation environment
Cell scale	1 m x 1 m in x/y	Coordinate mapping to Gazebo
Level spacing	2 m	Vertical separation between floors
LiDAR range	0.1 m to 8.0 m	Obstacle detection and mapping
Horizontal rays	180	Full 360-degree scan coverage
Offline vertical rays	32	3-D scan volume in offline mode
Fixed evaluation seed	20260607	Reproducible mission run

Results

The system was evaluated with a fixed-seed offline simulation. The cave generator used CAVE_SEED = 20260607 and produced a 64 x 32 x 4 grid. The selected start cell was (37, 18, 3), and the selected end cell was (32, 27, 0). The initial A* forward path contained 22 cells. After reaching the goal, the return planner produced an 18-cell path back to the start.

The full mission completed successfully. The robot reached the start cell again after 294 total steps. This demonstrates that the complete offline loop can generate a cave, plan a path, update the map during travel, and close the round trip. The run also showed waypoint skipping and replanning behavior, which indicates that recovery logic is active rather than only following a static ideal path.

The result should be interpreted as a simulation result rather than a hardware validation. The environment is generated, the robot model is simplified, and the offline run uses simulated LiDAR. However, the result is meaningful for the course objective because it demonstrates a working autonomous system simulation with planning, localization, mapping, and mission-level behavior.

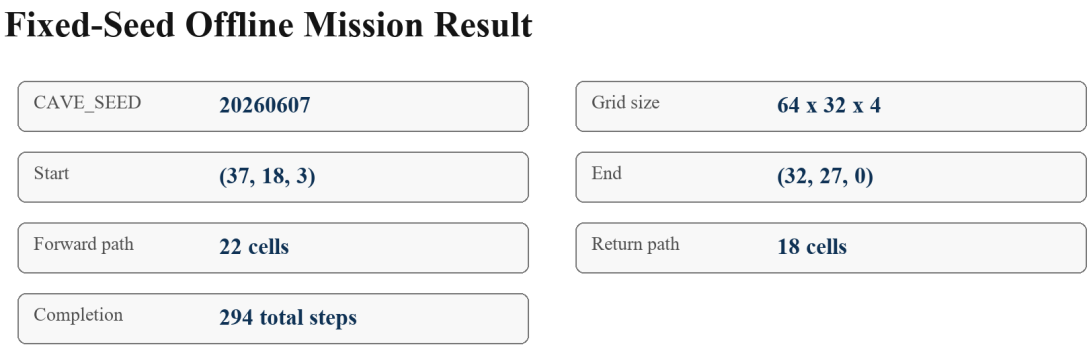

Figure 3

Fixed-seed offline simulation result used for reproducibility.

Discussion

The strongest aspect of the Cave Robot project is integration. A* planning, EKF localization, occupancy-grid mapping, procedural world generation, and ROS 2/Gazebo simulation are connected into one mission rather than shown as separate exercises. This gives the project a clear robotics workflow: generate an environment, plan, move, sense, map, and replan.

The main limitation is that the forward journey assumes a full map is known before navigation begins. This is acceptable for testing path planning and simulator integration, but it weakens the claim of autonomous exploration. A more realistic exploration system would plan on the SLAM map from the start, choose frontiers or information-gain targets, and update its route as new obstacles are discovered.

A second limitation is localization realism. The documentation and code indicate that the particle-filter SLAM structure exists, but map updates are stabilized using the trusted EKF/Gazebo pose. This makes the map more reliable in simulation, but it means the project is closer to mapping with a known pose estimate than to full independent SLAM. Future work should replace Gazebo pose correction with scan matching or another sensor-based localization method.

A third limitation is related to implementation requirements. The project is implemented primarily in Rust with ROS 2 and Gazebo tooling. If the course assessment strictly requires Python implementation, a Python demonstration wrapper or a Python version of the navigation loop should be added. The current implementation is technically strong, but the language choice should be explained honestly during submission.

Conclusion

Cave Robot demonstrates an autonomous indoor navigation system for cave-like environments. The project combines procedural generation, clearance-aware A* path planning, EKF-based localization, LiDAR occupancy-grid mapping, and round-trip mission execution. A reproducible offline simulation completed the start-to-goal-to-start task in 294 steps, supporting the claim that the core simulation is functional.

The project is best understood as a classical robotics integration study. It does not yet solve fully online exploration, hardware-grade SLAM, or real sensor uncertainty, but it establishes a working foundation for those extensions. The next improvements should focus on planning the forward leg from the SLAM map, using D* Lite for live replanning, replacing pose correction with scan matching, and adding a Python-facing demonstration layer if required by the course specification.

References

Elfes, A. (1989). Using occupancy grids for mobile robot perception and navigation. Computer, 22(6), 46-57. https://doi.org/10.1109/2.30720

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100-107. https://doi.org/10.1109/TSSC.1968.300136

Koenig, S., & Likhachev, M. (2002). D* Lite. Proceedings of the AAAI Conference on Artificial Intelligence, 16(1), 476-483.

Open Robotics. (2026). Gazebo documentation. https://gazebosim.org/docs/

Open Robotics. (2026). ROS 2 documentation. https://docs.ros.org/

Pugachuk, A., & Kullakshi, A. (2026). Cave Robot [Computer software]. GitHub. https://github.com/somethim/cave-robot

RogueBasin. (n.d.). Cellular automata method for generating random cave-like levels. https://www.roguebasin.com/index.php/Cellular_Automata_Method_for_Generating_Random_Cave-Like_Levels

Thrun, S., Burgard, W., & Fox, D. (2005). Probabilistic robotics. MIT Press.

Welch, G., & Bishop, G. (2006). An introduction to the Kalman filter. University of North Carolina at Chapel Hill.

Appendix A: Reproducibility Notes

The fixed-seed run reported in this paper used the repository's offline mode. The sequence was: set CAVE_SEED=20260607, generate cave.json, and run the robot executable against that cave file. The robot log reported a 64 x 32 x 4 cave, start cell (37, 18, 3), end cell (32, 27, 0), a 22-cell initial forward path, an 18-cell return path, and mission completion at the start cell after 294 total steps.

The project tests were executed with cargo test --workspace. The workspace reported passing tests for pathfinding and SLAM, including A* path discovery, no-path handling, D* Lite path discovery, D* Lite replanning, SLAM initialization, SLAM prediction, and SLAM map update.

Appendix B: Suggested Live Demonstration Script

1. Open the repository and show the architecture: generator, robot, pathfinding, slam, kalman, and ROS packages.
2. Run the generator with a fixed seed and show the produced cave.json or printed cave slices.
3. Start the offline robot run and point out the phase transition from forward navigation to return navigation.
4. In Gazebo mode, show the cave world, start and end markers, LiDAR-equipped drone, and motion in the generated environment.
5. End by showing the mission-complete log and explaining the current limitations: known forward map, simulated sensing, and partial SLAM localization.